

`Nullable<T>` requires that `T` be a struct (i.e., you can only mark value types as nullable objects).

```
public class MyNullable<T> where T : struct
{
    public T Value { get; private set; }
    public MyNullable(T t)
    {
        Value = t;
    }
}
```

Frequently, we wish to create new instances of our type parameters. However if you think about it, this really isn't possible because classes may specify any number of parameters for their constructors. At best, we can hope that our candidate classes or structs implement a default constructor. Luckily, C# allows us to specify that our type parameter can only be instantiated by a type that has a default constructor. This is useful for implementing a generic factory method.

```
public static class GenericFactory<T> where T : new()
{
    public static T Make() { return new T(); }
}
```

4.3 Type Erasure

Observe that generics actually do not affect the execution of your program as all of their processing occurs during type-checking/compile time. Thus a reasonable question to ask is do we actually need to keep around information about the generic's type parameters at runtime?

Removing this information so that it does not appear at runtime is called type erasure. Java's generics are type erased at compile time and do not appear at runtime. Thus, two instantiations of the same generic, e.g., `ArrayList<Integer>` and `ArrayList<String>` appear to be the same object when viewed with reflection or other type-inspection tools. This was due to the desire for Java bytecode using generics to be backwards compatible with older versions of the JVM (this reason was quickly rendered moot as changes to the language broke backwards compatibility, but the implementation of generics remained the same).

How does this work? Here is a simplified generic implementation of an `ArrayList` and its corresponding type erased variant for a particular instantiation.